

Livrable 1 : Note de Cadrage et Plan de Développement

Projet : Do It Yourself Privacy Guard (DIYPG)

Équipe projet : NAPOLETANO Ethan, BOUSSEKSOU Dorian, MOKHTARI Tahar,
BOILAT Clément

1 Analyse et Objectif du Sujet

Le projet DIYPG consiste à créer en C une application de sécurité cryptographique inspirée du logiciel Gnu Privacy Guard (GPG). L'objectif n'est pas simplement de reconstruire des algorithmes à partir des documents et codes fournis, mais de construire une solution complète et utilisable structurée autour de quatre piliers (détaillés dans le sujet) :

- **Confidentialité** : Chiffrement de messages et fichiers via l'algorithme RSA.
- **Authenticité et Intégrité** : Calcul de hashage et signatures électroniques.
- **Ergonomie et Gestion** : Un interprète de commandes et un annuaire de clés.
- **Confiance décentralisée** : Une Blockchain simulant une autorité de certification pour garantir la validité des clés publiques.

2 Ce que nous allons programmer

Phase 1 : Implémentation RSA et Outils de Base

- **1.0 Arithmétique modulaire** : Implémentation du crible d'Ératosthène, du test de primalité, de l'algorithme d'Euclide étendu et génération de clés RSA sur 64 bits.
- **1.1 – 1.2 Chiffrement et Fichiers** : Chiffrement octet par octet puis lecture/écriture de messages dans des fichiers binaires.
- **1.3 Base64 et Analyse de capacité** : Encodage/décodage binaire/Base64. Analyse du dépassement de capacité des `uint64_t` selon la constante `MAX_PRIME`.

Phase 2 : Chiffrement par Blocs et Bibliothèque GMP

- **Chiffrement par blocs** : Regroupement des octets par blocs de 4 octets avec conversion little-endian.
- **Intégration GMP** : Utilisation de la bibliothèque multiprécision GMP pour traiter les grandes puissances modulaires sans débordement de mémoire avec précision.
- **Échange de clés** : Procédure de transmission des clés publiques au format Base64 entre membres de l'équipe pour tester.

Phase 3 : Interprète de Commandes et Annuaire de Contacts

- **Boucle REPL** : Shell interactif assurant la saisie et l'exécution des commandes utilisateur.
- **Gestion du trousseau (V1.1)**
- **Annuaire de contacts (V2.0)** : Gestion des identifiants et suppression/ajout sélective de clés.
- **Protection** : Sauvegarde chiffrée et protégée sur disque du fichier d'annuaire.

Phase 4 : Signatures Numériques et Hachage

- **Empreinte numérique** : Calcul du digest SHA-256 d'un fichier (en clair ou chiffré).

- **Signature RSA** : Chiffrement du digest avec la clé privée de signature de l'émetteur.
- **Vérification d'authenticité** : Déchiffrement du digest avec la clé publique du signataire et comparaison avec l'empreinte recalculée.

Phase 5 : Registre d'Authentification des Clés par Blockchain

- **Structure de données** : Blocs chaînés par hash SHA-256 et liste de transactions.
- **Arbre de Merkle** : Calcul de la racine de Merkle pour sceller les transactions de chaque bloc.
- **Proof of Work** : Algorithme de minage imposant un critère de zéros initiaux selon une difficulté paramétrable.
- **Tokens & Événements** : Gestion de comptes (100 tokens utilisateur, 1000 institution) et enregistrement des événements.
- **Contrôle d'intégrité** : Bilan complet vérifiant la continuité des hashes depuis le bloc Génésis.

3 Architecture Logicielle et Découpage Modulaire

Pour garantir la logique et la bonne gestion de notre code, l'application est découpée en modules indépendants :

Module (Fichiers)	Fonctions Principales	Bibliothèques
rsa_core.c / .h	Génération de clés RSA, Miller-Rabin, Bézout, blocs de 4 octets et calculs GMP.	stdint.h, gmp.h
other_base64.c / .h	Encodage et décodage Base64 pour tampons mémoire et fichiers.	stdlib.h
keyring.c / .h	Gestion du trousseau personnel, annuaire V1/V2 et sauvegarde chiffrée.	rsa_core.h
sha256_sign.c / .h	Calcul des digests SHA-256, signature d'empreintes et vérification.	sha256.h
blockchain.c / .h	Structures de blocs/transactions (bc_rsa.h), Arbre de Merkle, minage et bilan.	sha256.h
cli_interpreter.c / .h	Boucle, parsing de la ligne de commande et appel des sous-systèmes.	Tous les modules
Makefile	Automatisation de la compilation du projet et gestion des dépendances.	Make
README.md	Documentation du projet, manuel d'utilisation de l'interprète de commande et bilan des tests d'overflow sur MAX_PRIME.	Markdown

4 Répartition des Tâches entre membres d'équipe

Chaque membre de l'équipe prend en charge un module principal tout en participant aux phases d'intégration :

Membre	Rôle Majeur	Modules et Tâches Attribuées	Implication aux Jalons CC
BOUSSEKSOU Dorian	Responsable de l'interprète de commande, Annuaire et REPL	Développement de l'interprète de commande (<code>cli_interpreter.c</code> et <code>main.c</code>), gestion du trousseau V1.1/V2.0 (<code>keyring.c</code>), sauvegarde chiffrée, <code>Makefile</code> et <code>README</code> .	CC1 (Note de cadrage), CC3 (Interprète), CC4 (Démonstration globale).
NAPOLETANO Ethan	Responsable socle RSA et GMP	Arithmétique RSA (Rabin, Bézout), découpage par blocs de 4 octets (<code>int2char.c</code>), intégration de la bibliothèque GMP et analyse d'overflow avec <code>MAX_PRIME</code> .	CC1 (Planning), CC2 (Primitives RSA et tests <code>MAX_PRIME</code>), CC3 (Blocs 4 octets et GMP).
MOKHTARI Tahar	Signatures et Base64	Hachage SHA-256, signature et vérification (<code>signature.c</code> , <code>sha256.c</code>)	CC2 (Base64), CC3 (Annuaire), CC4 (Signatures SHA-256).
BOILAT Clément	Responsable Blockchain et Certification	Blockchain, arbre de Merkle et minage (<code>blockchain.c</code>)	CC2 (Tests de chiffrement), CC4 (Blockchain, Merkle, minage et bilan).

5 Échéancier Prévisionnel et Jalons de CC

Le calendrier de réalisation s'étend de septembre à décembre 2026 et s'articule autour des 4 évaluations du projet :

Période	Phase du Projet	Objectifs et Livrables	Jalon
Semaine 1 (Fin Septembre – 04 Octobre)	Cadrage et Orga	Note de cadrage, découpage modulaire, répartition des tâches et planning prévisionnel.	CC1 → Rendu du dossier prévisionnel (4 octobre)
Semaine 2 (05 Octobre – 12 Octobre)	Phase 1	Primitives RSA, tests de primarité, conversion Base64, analyse d’overflow avec MAX_PRIME et README.	CC2 → Rendu du code de la Phase 1 et Recette en présentiel (12 octobre)
Semaines 3 – 5 (Mi-Octobre – Début Novembre)	Phase 2 (GMP et Blocs) et Phase 3 (Interprète)	Chiffrement par blocs de 4 octets avec GMP, interprète de commandes (les deux versions), annuaire de clés et sauvegarde chiffrée.	CC3 → Rendu des Parties 2 et 3 et Recette en présentiel (mi-novembre)
Semaines 6 – 10 (Novembre – Début Décembre)	Phase 4 (Signatures) et Phase 5 (Blockchain)	Hachage SHA-256, signatures numériques, structures Blockchain, Arbre de Merkle, minage et événements. Intégration globale	<i>Voir ci-dessous</i>
Semaine 11 (Mi-Décembre)	Recette Finale & Démonstration	Analyse mémoire et finalisation du Makefile et du README.md, recette globale du logiciel et présentation.	CC4 → Dépôt du programme complet et Présentation finale

6 Analyse des Risques et Solution

Risque Identifié	Impact Potentiel	Solution
Dépassement de capacité (<code>uint64</code>)	Calculs erronés à cause de la puissance modulaire en Phase 1.	Tests rigoureux sur la constante MAX_PRIME et bascule directe vers GMP en Phase 2.
Fuites mémoire (Pointeurs C)	Instabilité lors de la manipulation des listes chaînées ou d’objets GMP.	Implémentation de fonctions de libération (type <code>free</code>) à chaque fois et vérification avec Valgrind à chaque phase.
Erreurs de compilation GMP	Échec de construction du projet sur l’environnement d’évaluation.	Rédaction d’un Makefile très exhaustif incluant explicitement l’option <code>-lgmp</code> et documentation dans le README.
Incohérence Blockchain	Rupture du chaînage ou de l’arbre de Merkle lors de l’ajout de blocs.	Développement d’une fonction de bilan automatisée vérifiant la chaîne depuis le bloc Génésis.